Green University of Bangladesh
Department of Computer Science and Engineering (CSE)

Faculty of Sciences and Engineering

Semester: (Spring , Year:2026), B.Sc. in CSE (Day)

LAB REPORT NO: 3

Course Title: Artificial intelligence

Course Code: CSE 316 **Section:** 232-D5

+

Lab Experiment Name: Implementation of Iterative Deepening Depth First Search (IDDFS) and A* Search Algorithm

Student Details

	Name	ID
1.	Rukonuzzaman Topu	232002280

Submission Date : **3/30/2026**

Course Teacher's Name : Babe Sultana

[For Teachers use only: Don't Write Anything inside this box]

Lab Report Status

Marks: …… **Signature:.....**

Comments:..... **Date:.....**

1. Introduction:

Title: Implementation of Iterative Deepening Depth First Search (IDDFS) and A* Search Algorithm

Problem Summary: The experiment involves implementing two important search algorithms. First, Iterative Deepening Depth First Search (IDDFS) is used to traverse a dynamically created graph and find a path from a source node to a destination node. Second, the A* search algorithm is implemented to find the optimal path using heuristic values while printing the queue at each step.

2. OBJECTIVES/AIM

1. To implement IDDFS traversal on a graph
2. To find a path from source to destination using IDDFS
3. To implement A* search algorithm
4. To understand heuristic-based search techniques
5. To compare uninformed and informed search algorithms

3. PROCEDUR

Algorithm for IDDFS:

- Start with depth = 0
- Apply Depth Limited Search (DLS)
- If goal is found → stop
- Otherwise, increase depth and repeat
- Continue until goal is found or max depth reached

Algorithm for A* Search:

- Initialize a priority queue
- Insert start node with cost
- Select node with lowest cost (f = g + h)
- Expand neighbors 5. Update cost and path
- Repeat until goal is reached
- Print queue at each step

4.Implementation:

Overview:

- ❖ IDDFS implemented using recursion and depth-limited search
- ❖ Graph taken dynamically as input from user
- ❖ A* implemented using priority queue (heapq)
- ❖ Heuristic values used to guide search

Important Details:

- ❖ IDDFS combines DFS with depth limitation
- ❖ A* uses: ➢ g(n) = path cost ➢ h(n) = heuristic ➢ f(n) = g(n) + h(n)
- ❖ Queue state printed in A* for each iteration

Q1. Write a program to perform IDDFS traversal on a dynamic graph input from the user.

Code :

1st:

```
def dls(graph, node, depth, visited):
    if depth < 0:
        return

    print(node, end=" ")
    visited.add(node)

    for neighbor in graph[node]:
        if neighbor not in visited:
```

```
            dls(graph, neighbor, depth - 1, visited)



def iddfs(graph, start, max_depth):
    print("\nIDDFS Traversal:")

    for depth in range(max_depth + 1):
        print(f"\nDepth {depth}: ", end="")
        visited = set()
        dls(graph, start, depth, visited)



graph = {}

n = int(input("Enter number of nodes: "))

for i in range(n):
    node = input(f"Enter node {i+1}: ")
    neighbors = input(f"Enter neighbors of {node} (space separated): ").split()
    graph[node] = neighbors

start = input("Enter starting node: ")
max_depth = int(input("Enter maximum depth: "))

iddfs(graph, start, max_depth)
```

OUTPUT:

```
$ C:/Users/Admin/.local/bin/python3.14.exe "d:/A
rtificial intelligence/lab report/python Rukonuz
zaman Topu 280/lab report 3/1st.py"
Enter number of nodes: 4
Enter node 1: A
Enter neighbors of A (space separated): B C
Enter node 2: B
Enter neighbors of B (space separated): D
Enter node 3: C
Enter neighbors of C (space separated): D
Enter node 4: D
Enter neighbors of D (space separated): E
Enter starting node: A
Enter maximum depth: 2

IDDFS Traversal:

Depth 0: A
Depth 1: A B C
Depth 2: A B D C
```

Q2. Write a program to find the path from source to destination using IDDFS.

CODE:

```
# Depth Limited Search (DLS)

def dls(graph, current, goal, depth, path, visited):

    if depth < 0:

        return None



    path.append(current)
```

```
    visited.add(current)

    if current == goal:
        return path.copy()

    for neighbor in graph.get(current, []):
        if neighbor not in visited:
            result = dls(graph, neighbor, goal, depth - 1, path, visited)
            if result:
                return result

    path.pop()
    return None



# IDDFS Function
def iddfs(graph, start, goal, max_depth):
    for depth in range(max_depth + 1):
        visited = set()
        path = []

        result = dls(graph, start, goal, depth, path, visited)

        if result:
            print("\nPath found at depth:", depth)
```

```
            print("Path:", " -> ".join(result))
            return

    print("\nNo path found!")
# Main Program
graph = {}

n = int(input("Enter number of nodes: "))

for i in range(n):
    node = input(f"Enter node {i+1}: ")

    if node in graph:
        print("Node already exists! Enter unique node.")
        continue

    neighbors = input(
        f"Enter neighbors of {node} (space separated): "
    ).split()

    graph[node] = neighbors

start = input("Enter start node: ")
goal = input("Enter goal node: ")
# Validation
```

```
if start not in graph or goal not in graph:
    print("Error: Start or Goal node not found!")
    exit()
max_depth = int(input("Enter maximum depth: "))


iddfs(graph, start, goal, max_depth)
```

Output:

```
Admin@DESKTOP-2JIMCMR MINGW64 /d/Artificial intell
igence/lab report/python Rukonuzzaman Topu 280/lab
 report 3 (main)
(space separated): B C
Enter node 2: B
Enter neighbors of B (space separated): D
Enter node 3: C
Enter neighbors of C (space separated): E
Enter node 4: D
Enter neighbors of D (space separated):
Enter node 5: E
Enter neighbors of E (space separated):
Enter start node: A
Enter goal node: E
Enter maximum depth:

Path found at depth: 2
Path: A -> C -> E

Admin@DESKTOP-2JIMCMR MINGW64 /d/Artificial intell
igence/lab report/python Rukonuzzaman Topu 280/lab
 report 3 (main)
$
```

Q3. Write a program to perform the A* Algorithm on the following figure. Explicitly Print the queue at each step.

CODE:

```
import heapq


def a_star(graph, heuristic, start, goal):
    # Priority Queue: (f_cost, node, path, g_cost)
    queue = []

    heapq.heappush(queue, (heuristic[start], start, [start], 0))

    visited = set()

    while queue:

        # Print queue at each step
        print("\nCurrent Queue:")
        for item in queue:
            print(item)

        f, current, path, g = heapq.heappop(queue)

        if current == goal:
            print("\nGoal Reached!")
            print("Path:", " -> ".join(path))
            print("Total Cost:", g)
```

```
            return

        if current in visited:
            continue

        visited.add(current)

        for neighbor, cost in graph[current]:
            if neighbor not in visited:

                new_g = g + cost
                new_f = new_g + heuristic[neighbor]

                heapq.heappush(
                    queue,
                    (new_f, neighbor, path + [neighbor], new_g)
                )

    print("No path found!")



# Graph
graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 1)],
    'C': [('F', 5)],
```

```
    'D': [],
    'E': [('F', 2)],
    'F': []
}

# Heuristic values
heuristic = {
    'A': 6,
    'B': 4,
    'C': 5,
    'D': 2,
    'E': 1,
    'F': 0
}

start = 'A'
goal = 'F'

a_star(graph, heuristic, start, goal)
```

Output :

```
 report 3 (main)
$ C:/Users/Admin/.local/bin/python3.14.exe "d:/Art
ificial intelligence/lab report/python Rukonuzzama
n Topu 280/lab report 3/3RD.PY"

Current Queue:
(6, 'A', ['A'], 0)

Current Queue:
(5, 'B', ['A', 'B'], 1)
(8, 'C', ['A', 'C'], 3)

Current Queue:
(3, 'E', ['A', 'B', 'E'], 2)
(8, 'C', ['A', 'C'], 3)
(6, 'D', ['A', 'B', 'D'], 4)

Current Queue:
(4, 'F', ['A', 'B', 'E', 'F'], 4)
(8, 'C', ['A', 'C'], 3)
(6, 'D', ['A', 'B', 'D'], 4)

Goal Reached!
Path: A -> B -> E -> F
Total Cost: 4

Admin@DESKTOP-2JIMCMR MINGW64 /d/Artificial intell
igence/lab report/python Rukonuzzaman Topu 280/lab
 report 3 (main)
$
```

6. ANALYSIS AND DISCUSSION:

Both IDDFS and A* algorithms were successfully implemented and produced correct results. IDDFS was able to find the path by gradually increasing the depth limit, which makes it memory efficient but sometimes slower. On the other hand, A* search performed more efficiently by using heuristic values to guide the search toward the goal, resulting in faster and optimal path finding. Some challenges were faced in managing recursion in IDDFS and maintaining the priority queue in A*, but these were resolved through debugging. Overall, the experiment helped in understanding the difference between uninformed and informed search algorithms, and all objectives were successfully achieved.